OOPJ Unit III: Unified Faculty Study and Exploration Guide

java.lang.Object, inherited-member access, and final

Purpose: one integrated reference for faculty-level understanding and teaching practice. It combines the complete explanation, contextual diagrams, Unit II prerequisite visuals, runnable-code map, guided investigations, source/resource audit, misconceptions, and rehearsal tasks in a single reading sequence.

Technical baseline: Java Language Specification (JLS), Java SE 27; examples validated with javac 21.0.12.1. The examples use ordinary syntax suitable for JDK 17 and later. Check the actual JDK on the lab machine before teaching.

How this guide fits your existing material

Your existing OOPJ_Unit_III_Inheritance_Fundamentals_Faculty_Guide introduces Object, inherited-member access, and final. This guide is a **teaching-and-mastery companion**, not a replacement for that broader inheritance chapter. Use the previous guide for the full inheritance narrative, then use this one to rehearse the difficult distinctions and demonstrate them confidently.

Work through the guide in five passes:

1. **Understand:** read the core explanation without opening the IDE.
2. **Reconstruct:** draw the hierarchy, member-access matrix, or object/reference diagram from memory.
3. **Predict:** answer every code-tracing question before running an example.
4. **Verify:** compile and execute the valid examples; separately compile the intentionally invalid examples and explain the rule causing each error.
5. **Teach:** rehearse the short lesson sequence and record yourself explaining the concept to an imagined large classroom.

Throughout, label statements mentally as one of four kinds:

- **Language rule:** what Java requires, as specified by the JLS.
- **API contract:** what a method such as Object.equals promises.
- **Teaching model:** a simplified diagram used to make reasoning visible.
- **Design advice:** a recommendation for understandable, maintainable code, not necessarily a compiler rule.

Keeping these categories separate is a mark of strong faculty-level explanation.

What is integrated in this single guide: three prerequisite visuals from the Unit II reference package; five topic-specific diagrams placed beside the relevant explanations; model code, expected outputs and negative examples; a focused source/library-use audit; guided investigation questions; and a daily study/rehearsal plan.

Visual study map: use these diagrams while exploring the rules

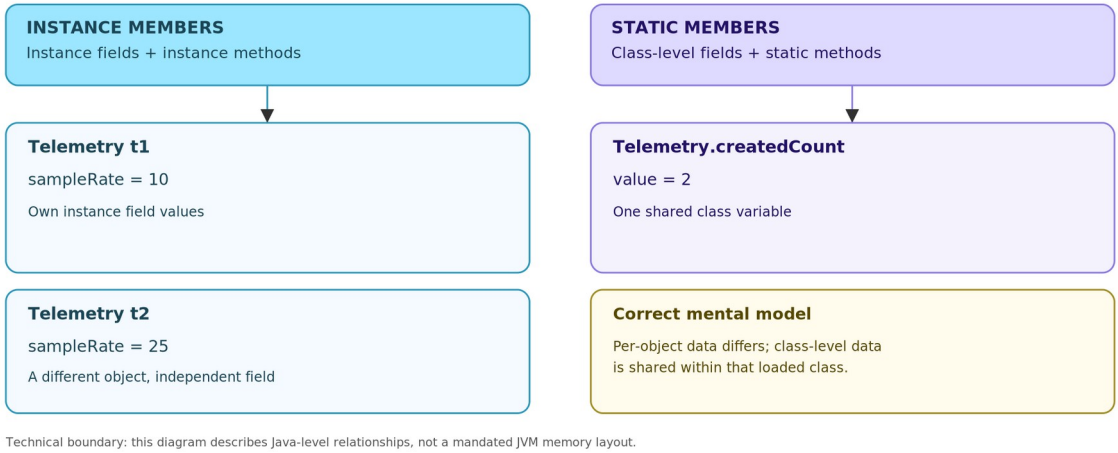
Visual 1: Object hierarchy and common methods. Visual 5: Object method contracts and equals signatures. Visual 2: member inheritance versus access. Visual 3: protected receiver restriction across packages. Visual 4: final class, method and variable. The individual SVG and PNG files are also supplied in the visual companion ZIP.

Bridge from Unit II: rebuild the mental models first

These three figures are not inheritance diagrams. They are the prerequisite mental models needed to understand inherited members correctly: a class distinguishes instance and class-level state; declaration is not object creation; and assignment of a reference can produce aliases. Study them in order before proceeding to Object and inherited access.

Figure 3.1 | Class members: per-object state and class-level state

Keep instance state separate from shared class state. Methods define behavior; they are not copied into each object.



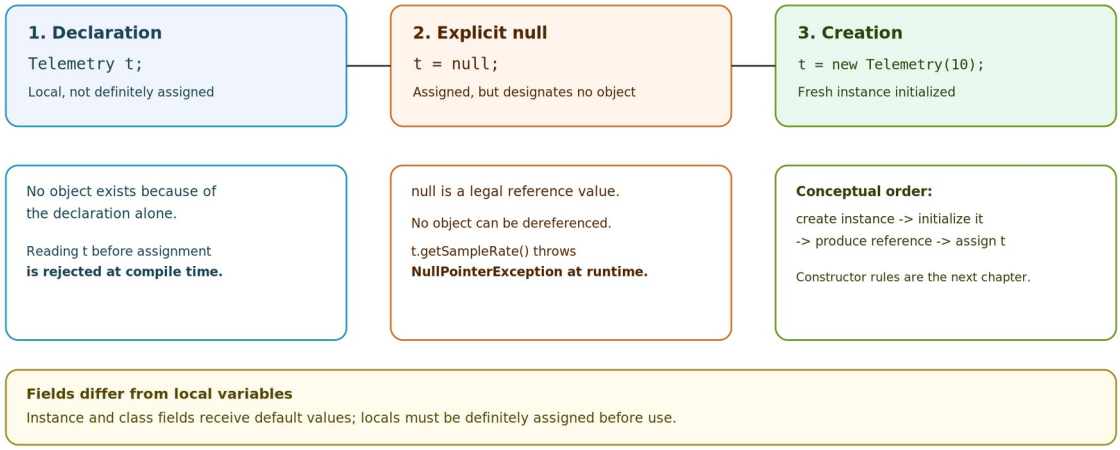
Technical boundary: this diagram describes Java-level relationships, not a mandated JVM memory layout.

Bridge figure U2-1. Instance state differs per object; static state is class-level.

Ask: Which value belongs to each object? Which value is shared by the class? Why does this not mean methods are copied into each object?

Figure 3.2 | Declaration is not object creation

The name is a variable; the object is a separate runtime entity. A successful new expression yields a reference value.

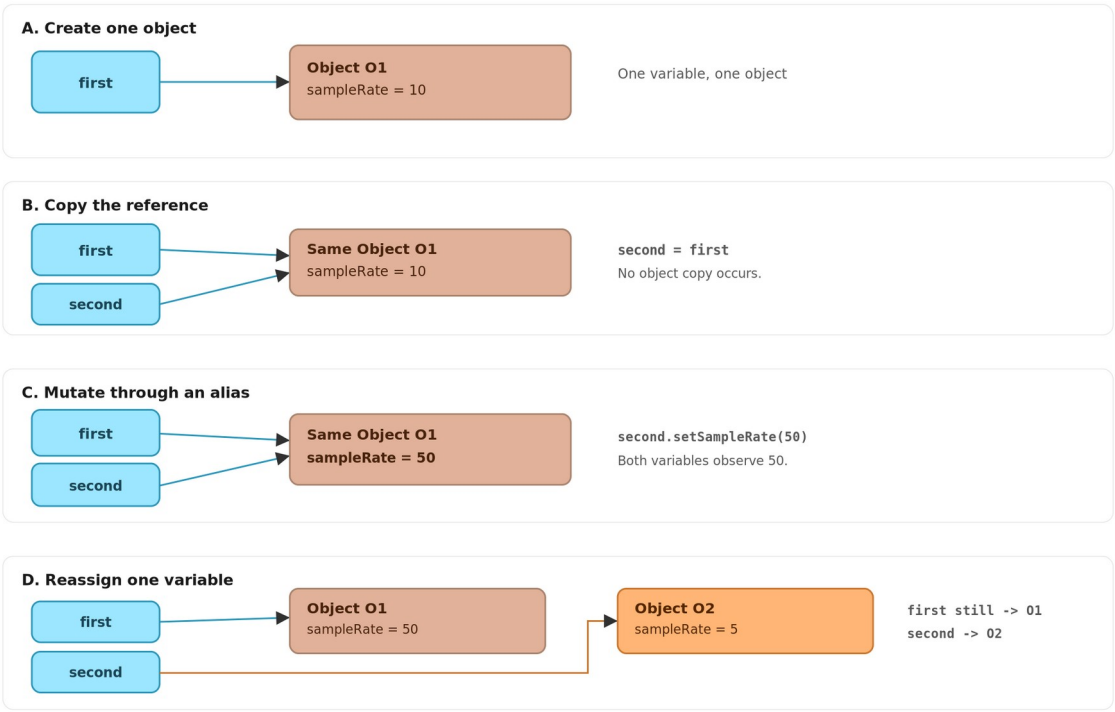


Bridge figure U2-2. A variable declaration does not by itself create an object.

Ask: What exists after declaration? What changes after new? Where does the constructor fit in the sequence?

Figure 3.3 | Aliasing, mutation and reassignment

The same reference value can be stored in two variables. Mutating the object is different from changing a variable.



Bridge figure U2-3. Assignment copies a reference value; aliases can name the same object.

Ask: What happens after mutation through one alias? What happens after only one variable is reassigned?

Learning outcomes

By the end, you should be able to:

- Explain why ordinary Java classes are related to java.lang.Object and what that relationship lets a program do.
- Explain and demonstrate toString(), equals(Object), hashCode(), and getClass() without misleading students about memory addresses or identity.
- Distinguish a member being inherited from a member being accessible at a particular use site.
- Explain why constructors are not inherited and why private superclass fields are not directly inherited by a subclass.
- Apply package-private and protected access rules, including the cross-package protected receiver restriction.
- Distinguish final on a class, method, local variable, parameter, and reference-valued variable.
- Explain why a final reference does not automatically make the referenced object immutable.
- Predict and explain compiler errors involving final classes/methods and illegal protected access.
- Choose examples that expose misconceptions and test transfer to new examples.

1. Start with a purpose: why talk about `Object`?

1.1 The core idea

java.lang.Object is the root superclass of the ordinary Java class hierarchy. If a class declaration does not explicitly name a superclass, Object is its direct superclass (except Object itself and special class forms with specified superclass rules). An ordinary subclass is also indirectly related to Object through its superclass chain. [1, §8.1.4]

```
class Device { }
class Printer extends Device { }

public class ObjectRootDemo {
   public static void main(String[] args) {
      Printer p = new Printer();
      Object o = p;
      System.out.println(o.getClass().getSimpleName());
   }
}
```

Expected output

```
Printer
```

Read the two variables carefully:

- `p` has declared type `Printer` and refers to a `Printer` object.
- `o` has declared type `Object`, yet refers to the same object.
- `o.getClass()` reports the runtime class of the object, not the declared type of the variable.

The assignment `Object o = p;` is legal because a `Printer` is usable where an `Object` reference is expected. It does not create a second `Printer` object.

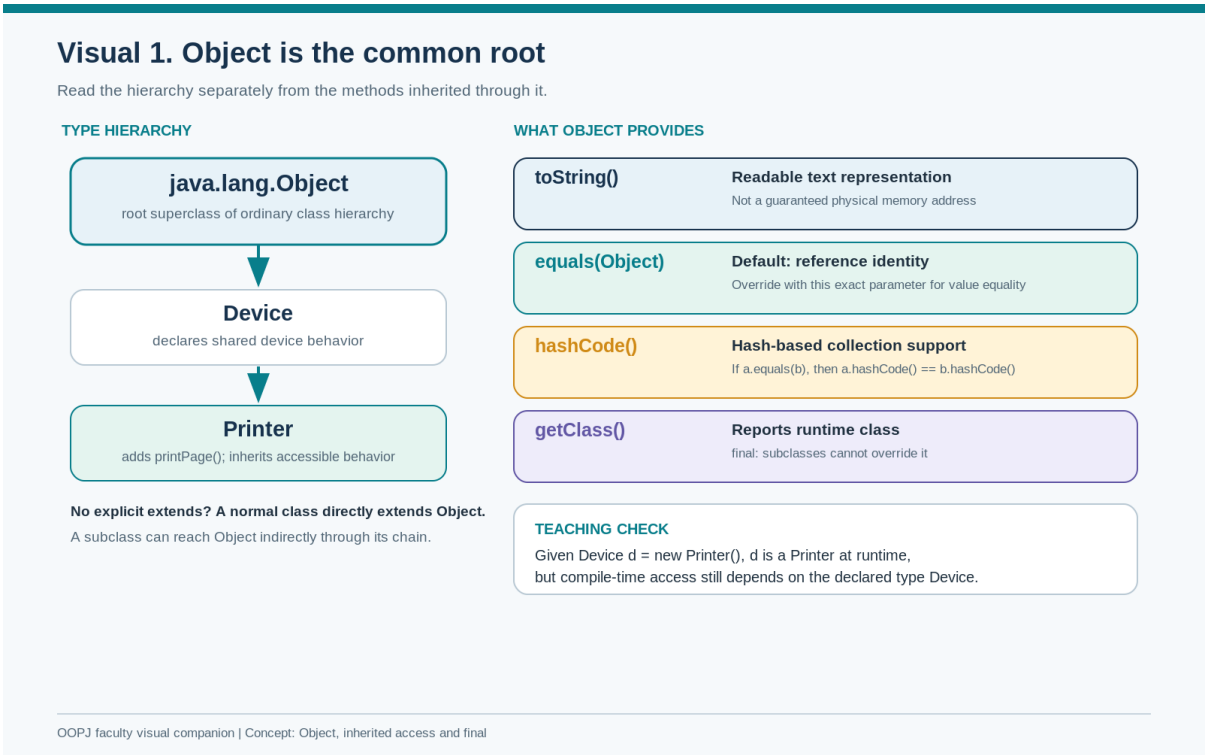
1.2 What `Object` gives us

The `Object` API supplies general operations that many classes can use or specialize. For the introductory inheritance lesson, focus on four methods:

Method	Core purpose	Teaching emphasis
`toString()`	Textual representation of an object	Helpful output and debugging; often overridden for meaningful domain data.
`equals(Object)`	Equality comparison according to the class's contract	Base implementation is identity-based; a class can define value equality.
`hashCode()`	Hash code used by hash-based data structures	Equal objects must have equal hash codes when `equals` is overridden consistently.
`getClass()`	Runtime class object	Can expose the actual class of the object; the method is `final`.

Do not try to teach the entire `Object` API at once. Methods such as `wait`, `notify`, and `notifyAll` require concurrency context. `clone()` has a more involved contract. `finalize()` is deprecated for removal and should not be presented as a recommended lifecycle technique. [3]

1.3 A classroom-friendly hierarchy explanation



Visual 1. Object is the root of the ordinary class hierarchy.

Board sequence:

```
Object
  |
Device
  |
Printer
```

Then ask:

1. “Which variable types can refer to a `Printer` object?”
2. “If the variable is declared as `Object`, can the compiler find `printPage()` on that variable?”
3. “What does `getClass()` tell us that the variable declaration does not?”

Expected reasoning:

- A `Printer` object can be referenced as `Printer`, `Device`, or `Object`.
- The compiler checks members using the compile-time type of the expression. `Object o = new Printer(); o.printPage();` is a compile-time error because `Object` does not declare `printPage()`.
- At runtime, `o.getClass()` identifies the actual runtime class, `Printer`.

Important boundary: runtime class information does not mean every subclass-specific method becomes directly callable through every superclass reference. The declared type still matters to compile-time member lookup.

2. The four high-value `Object` methods

2.1 `toString()`: text, not a memory address

A useful `toString()` should produce a concise, informative textual representation. The base `Object.toString()` implementation combines the runtime class name, an `@` character, and a hexadecimal representation related to the object's hash code. This result is **not a guaranteed physical memory address**, and it is not guaranteed to remain identical across executions. [3]

```
class Course {
    private final String code;
    private final String title;

    Course(String code, String title) {
        this.code = code;
        this.title = title;
    }

    @Override
    public String toString() {
        return "Course[code=" + code + ", title=" + title + "]";
    }
}
```

Teaching point: System.out.println(course) calls String.valueOf(course), which uses the object's string representation when the reference is non-null. An intentional toString() makes program output interpretable for both developers and learners.

Ask learners: "Does the output of the default toString() prove where the object is stored in RAM?"

Expected answer: No. It is a textual representation based on class name and hash-code-related information, not a promise to disclose a memory address.

2.2 equals(Object): identity versus value equality

The default Object.equals(Object) returns true only when the references identify the same object. A class can override equals(Object) to define domain-specific value equality. The method's contract includes reflexivity, symmetry, transitivity, consistency, and x.equals(null) being false for a non-null x. [3]

Consider this incomplete pair:

```
class Ticket {
    private final int number;

    Ticket(int number) {
        this.number = number;
    }

    // This is an overload, not an override of Object.equals(Object).
    public boolean equals(Ticket other) {
        return other != null && number == other.number;
    }
}
```

Why is this a faculty-level trap? The method signature equals(Ticket) is different from equals(Object). Calls where both expressions are statically typed as Ticket may select the overload, but a call through an Object reference resolves against equals(Object) and does not use this method as an override.

The reliable practice is to write @Override and implement the exact equals(Object) signature when overriding the base method. Example:

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof Ticket other)) return false;
    return number == other.number;
}
```

This example uses modern pattern matching for instanceof; for older course JDKs, write the explicit cast after the type check. This guide's runnable code uses JDK 17-compatible syntax.

2.3 hashCode(): the paired contract

Hash-based collections use hash codes to help locate candidate entries efficiently. The required relationship is:

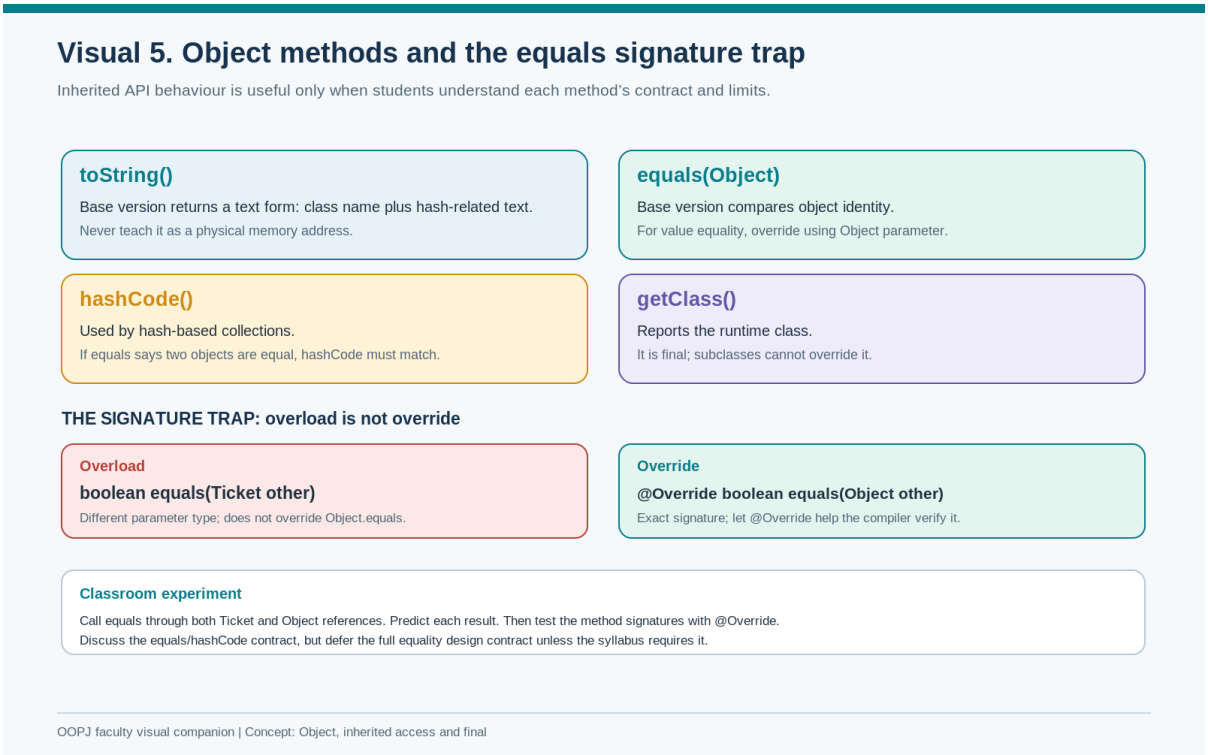
> *If two objects are equal according to `equals`, their `hashCode()` results must be equal.*

The converse is **not** required: two unequal objects may have the same hash code. A class that overrides `equals` should generally override `hashCode` consistently. [3]

Teaching example: two separate ticket objects can be value-equal even though `a == b` is false. If `equals` says that their ticket numbers make them equal, `hashCode` must honour that equality rule.

Do not tell students “different objects always have different hash codes.” That is false.

2.4 `getClass()`: runtime class, and a final method



Visual 5. The core Object methods and equals signature trap.

`getClass()` returns the runtime class of the object, as a `Class<?>` object. It is a `final` method in `Object`, so a subclass cannot override it. [3]

```
Object value = "Java";
System.out.println(value.getClass().getName());
```

Expected output:

```
java.lang.String
```

Notice that `value` is declared as `Object`, but the runtime object is a `String`. This is an excellent bridge to later study of overriding and dynamic dispatch. Keep those advanced dispatch rules in their designated chapter rather than attempting to teach all of polymorphism here.

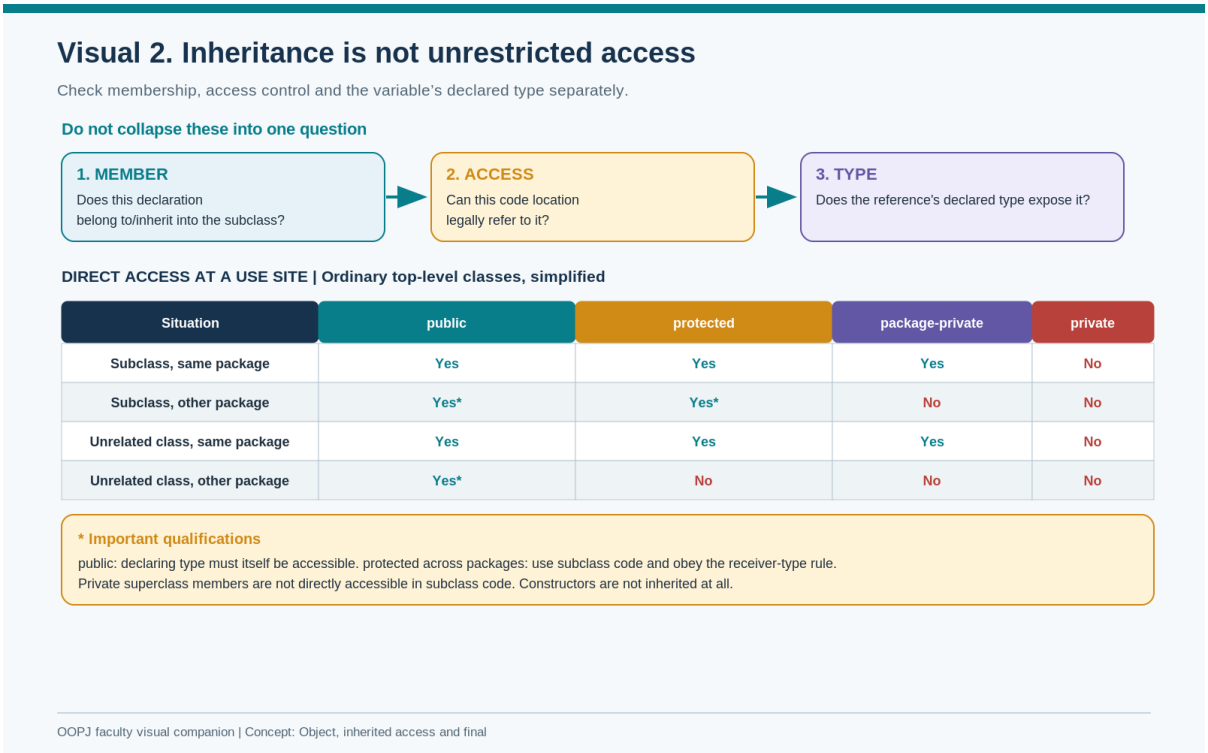
3. Inherited members: membership and accessibility are different questions

This distinction is one of the most important things to teach precisely.

- **Inheritance question:** Is the superclass declaration inherited as a member of the subclass under Java's rules?
- **Accessibility question:** Is this particular use of the declaration permitted from this location, given the declaration's access and the expression's type?

The JLS defines class members and inheritance rules; access control is specified separately. Private members are not inherited as subclass members. Fields and methods also have distinct inheritance and hiding/overriding rules. [1, §§8.2, 8.3, 8.4.8; 2, §6.6]

3.1 Practical member/access table



Visual 2. Inheritance, accessibility and static type are separate checks.

Superclass declaration	Same-package subclass	Subclass in a different package	Key caution
public field/method	Inherited and accessible	Inherited and accessible	The declaring type itself must also be accessible.
protected field/method	Inherited and accessible within package	Inherited; cross-package use must occur in subclass code and obey the qualifying-type rule for instance members	"Protected means any object can access it" is wrong.
Package-private field/method (no access modifier)	Inherited and accessible if otherwise applicable	Not inherited as an accessible member of the cross-package subclass	Package identity, not similar package names, controls package access.
private field/method	Not inherited as a member of the subclass	Not inherited as a member of the subclass	Superclass methods can provide controlled access to private state. A nested/nest access situation is a separate rule.
Constructor	Not inherited	Not inherited	A subclass constructor can invoke a superclass constructor; that is not constructor inheritance.

This table is a teaching summary, not a complete replacement for the JLS. Special cases involving overriding, hiding, nested classes and module/type accessibility require their own rules.

3.2 Minimal example: accessible behavior versus private state

```
class BaseRecord {
    public String id = "R-01";
    protected String status = "OPEN";
    String note = "same package only";
    private String secret = "internal";

    public String readSecret() {
        return secret;
    }
}

class SpecialRecord extends BaseRecord {
    public void inspect() {
        System.out.println(id);      // public: available
        System.out.println(status);  // protected: available here
        System.out.println(note);    // package-private: this example is same-package
        // System.out.println(secret); // compile-time error: private field is not inherited
        System.out.println(readSecret()); // public superclass operation exposes chosen behavior
    }
}
```

Expected output:

```
R-01
OPEN
same package only
internal
```

The last line does not mean `secret` became inherited or public. `readSecret()` executes in the superclass's access context and returns a value through an intentional public operation.

3.3 Constructors are not inherited

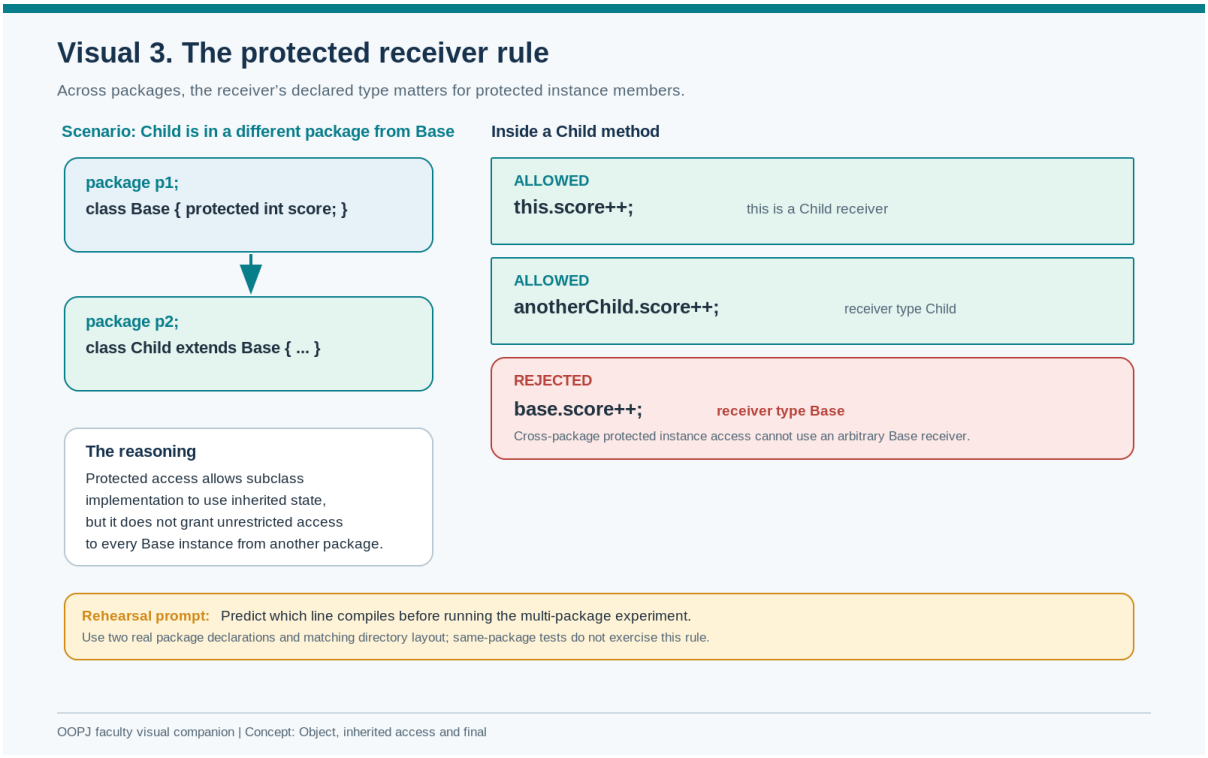
A subclass does not receive a superclass constructor as an inherited method. It can invoke an accessible superclass constructor during its own construction. Constructor chaining and initialization order belong to a subsequent `super`/constructors chapter. For this lesson, keep the message crisp:

> *A subclass inherits eligible members. Constructors initialize instances, but constructors are not members inherited like fields and methods.*

Do not confuse “the superclass constructor runs when a subclass object is constructed” with “the subclass inherited the constructor.” Those are different claims.

4. The difficult rule: `protected` across packages

4.1 The simple story is incomplete



Visual 3. Cross-package protected receiver restriction.

Within the same package, protected members are accessible under the package access path. Outside the package, use must occur within subclass code, and an instance field/method access is additionally constrained by the qualifying type: the expression used to access the member must have the subclass type `S` or a subtype of `S`. [2, §6.6.2.1]

This is why “a subclass can access any protected member through any superclass-typed object” is an unsafe teaching statement.

4.2 Valid and invalid receiver examples

Source layout:

```
src/
  alpha/
    Base.java
  beta/
    Child.java
```

alpha/Base.java

```
package alpha;

public class Base {
    protected int value = 10;
}
```

beta/Child.java

```
package beta;

import alpha.Base;

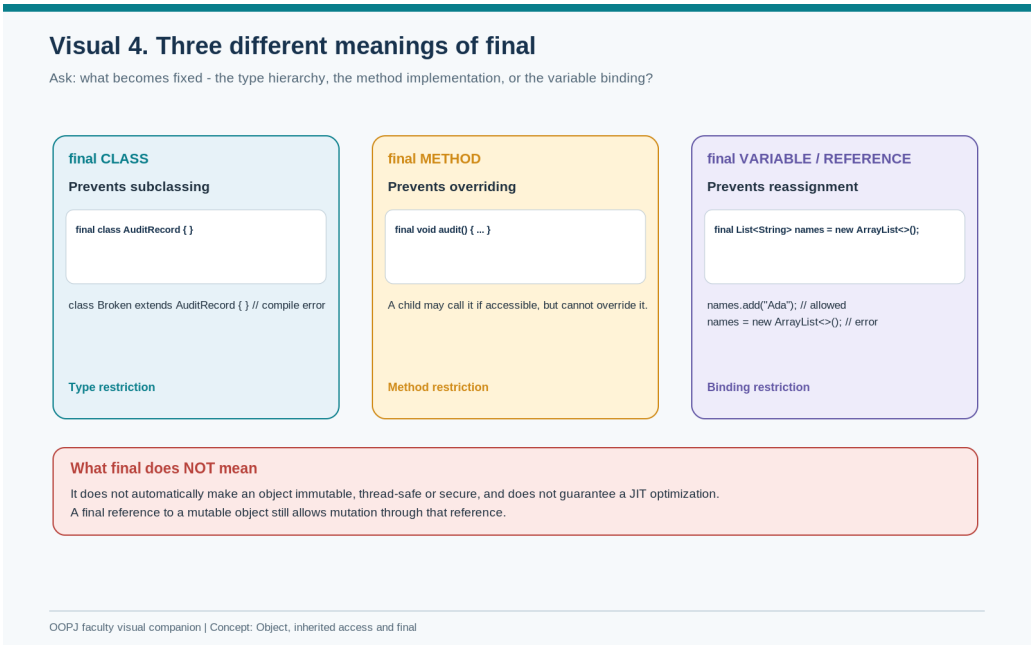
public class Child extends Base {
   public void show() {
      System.out.println(value); // permitted: inherited protected member
      Child otherChild = new Child();
      System.out.println(otherChild.value); // permitted: receiver type is Child

      Base plainBase = new Base();
      // System.out.println(plainBase.value); // compile-time error
   }
}
```

The commented line is intentionally invalid from this cross-package subclass context. The reason is not that value is never inherited. The issue is that plainBase has qualifying type Base, not Child or a subtype of Child, for this protected instance-member access.

Teaching procedure: have students classify each line before compiling: legal, illegal, and the exact rule. Only after every prediction should the class be compiled. Keep the two packages real in the folder layout; placing both classes in one package would fail to test this rule.

5. final: three different restrictions



Visual 4. The three different meanings of final.

The keyword final does not have one universal meaning. Its effect depends on the declaration it modifies.

Where final appears	What it prevents	What it does not automatically mean
Class: final class X	Subclassing that class	The class cannot be instantiated. It usually can be instantiated if it has an accessible constructor.
Instance method: final void f()	Subclasses overriding or hiding that method	The method is necessarily faster or inlined by the JVM.
Variable/field/parameter/local	Reassignment after definite assignment, subject to the declaration's rules	The referenced object is deeply immutable.

5.1 Final class

```
final class Receipt {
   private final int number;

   Receipt(int number) {
      this.number = number;
   }

   int getNumber() {
      return number;
   }
}
```

new Receipt(7) is legal. class SpecialReceipt extends Receipt {} is a compile-time error. A final class is not the same thing as an abstract class; the combination abstract final is illegal because it would be impossible to subclass the abstract class to complete it. [1, §8.1.1.2]

5.2 Final method

```
class Policy {
   public final void displayRule() {
      System.out.println("Apply the standard rule.");
   }

   public void displayNote() {
      System.out.println("Base note");
   }
}

class LocalPolicy extends Policy {
   // displayRule() cannot be overridden here.

   @Override
   public void displayNote() {
      System.out.println("Local note");
   }
}
```

A final method can be inherited and called normally. final constrains a subclass from replacing the method implementation by overriding/hiding it; it does not make the method private or inaccessible. A private method is not overridable in the normal inheritance sense, and methods declared in a final class cannot be overridden because no subclass can exist. [1, §8.4.3.3]

5.3 Final variable and final reference

```
class MutableLabel {
   String text;

   MutableLabel(String text) {
      this.text = text;
   }
}

public class FinalReferenceDemo {
   public static void main(String[] args) {
      final MutableLabel label = new MutableLabel("Draft");
      label.text = "Approved"; // allowed: mutates the referenced object
      System.out.println(label.text);

      // label = new MutableLabel("New"); // compile-time error: reassigns final variable
   }
}
```

Expected output:

```
Approved
```

A final reference variable cannot be reassigned after it has been assigned. But if it refers to a mutable object, the object's fields may still change when those changes are permitted by access rules. This is why **final reference** and **immutable object** must not be treated as synonyms.

Also distinguish:

- A final primitive variable cannot be assigned a new primitive value after assignment.
- A final reference variable cannot be assigned a different reference after assignment.
- The object referenced by a final reference may still be mutable.
- final is not a general promise of thread safety or deep immutability.

5.4 Is final a guaranteed optimization?

No. Do not teach “final makes the method faster” or “the compiler will inline every final method” as a language guarantee. The JLS notes that an implementation may inline a final method if it preserves semantics, but optimization choices are implementation/runtime matters, not a promise that every call will be inlined. [1, §8.4.3.3]

The primary language-level reason for final is to express a restriction on further subclassing, overriding/hiding, or reassignment, depending on the declaration.

6. Common misconceptions and the repair explanation

Misconception	Better correction to teach
toString() prints an object's physical memory address.	Its base implementation prints the runtime class name and hash-code-related text; this is not a guaranteed address.
equals(Ticket) overrides Object.equals.	It overloads the name with a different parameter type. Use equals(Object) and @Override for the override.
If two objects are unequal, their hash codes must differ.	Unequal objects may share a hash code. The required direction is: equal objects must have equal hash codes.
A subclass inherits every superclass member, including constructors and private fields.	Constructors are not inherited; private members are not inherited as subclass members. Other inheritance rules depend on member kind and access.
protected means any subclass can access the member through any instance.	Cross-package access must occur in subclass code, and instance-member receiver types are constrained.
A final class cannot be instantiated.	A final class can be instantiated if it has an accessible constructor; final prevents subclassing.
A final method cannot be called.	It can be called and inherited; a subclass cannot override/hide it.
A final reference makes the object immutable.	It prevents reassignment of that variable, not mutation of the referenced object.
All final methods are automatically inlined.	A runtime may optimize calls while preserving semantics; no categorical performance guarantee follows from the keyword.
If a field is private, no code can ever access it in any context.	A private field is not inherited by a subclass. Code within the declaring class and certain nest contexts may have private access; API methods can also provide intentional access.

7. A progressive teaching sequence

Use this sequence for a 20-minute explanation inside a larger inheritance lesson. Do not start by listing rules. Start with questions that establish a reason to care.

Phase 1: `Object` as the common reference type (4 minutes)

1. Draw `Object -> Device -> Printer`.
2. Show `Object x = new Printer();`.
3. Ask what `x.getClass()` reports and whether `x.printPage()` compiles.
4. Explain the contrast between the variable's declared type and the runtime object's class.

Check for understanding: ask the learner to explain why `getClass()` can identify `Printer` while `x` is declared as `Object`.

Phase 2: inherited member versus accessible member (6 minutes)

1. Display the four access levels.
2. Use `BaseRecord` and `SpecialRecord` to distinguish public/protected/package/private members.
3. Explain that private superclass state is not inherited as a subclass member.
4. Explain why a public or protected superclass method may expose a controlled operation on private state.
5. State that constructors are not inherited, even though superclass construction is part of subclass construction.

Check for understanding: give three lines of code and ask students to classify them as legal or illegal before compiling.

Phase 3: `protected` across packages (4 minutes)

1. Put `Base` in package `alpha` and `Child` in package `beta`.
2. Predict `value`, `child.value`, and `base.value` accesses.
3. Explain the receiver-type restriction in plain language.
4. Compile the legal case first and show the invalid line separately.

Check for understanding: ask why `new Base().value` is different from `new Child().value` inside `Child` code in the cross-package case.

Phase 4: the three meanings of `final` (5 minutes)

1. Show `final class Receipt`: can it be instantiated? Can it be extended?
2. Show a `final` method alongside an overridable method.
3. Show a `final` reference pointing to a mutable object; mutate a field, then explain why reassignment is prohibited.
4. Correct the “final always means faster” misconception.

Check for understanding: have learners complete the sentence “`final` prevents ___ here, but does not necessarily prevent ___.”

Phase 5: retrieve the rule (1 minute)

Ask students to explain one example without looking at the code. End with one prediction problem that combines reference type, inherited method access, and a final restriction.

8. Your focused 100-minute home-practice plan

This plan is specific to this guide's topics. Use the same method later for other Unit III chapters.

Time	Activity	Evidence to produce	Suggested tool
0-10 min	Recall Object, access levels, and meanings of final without notes	Short spoken explanation and list of uncertain points	ChatGPT / notebook
10-25 min	Study one section and verify one disputed rule	Annotated notes distinguishing language rule, API contract, and design advice	This guide + JLS/API
25-45 min	Reconstruct and run one code example; predict first, then verify	Compiler-tested program, expected output, and one variation	VS Code + JDK
45-55 min	Draw a hierarchy/access diagram and plan guiding questions	Board sequence with a misconception checkpoint	Miro / digital board
55-75 min	Deliver a 15-20 minute lesson segment standing up	Recorded video of voice, body, board, and code workflow	OBS or phone camera
75-90 min	Review the recording, then answer three viva questions aloud	Three observations tied to evidence and one corrected explanation	Gemini / ChatGPT Voice
90-100 min	Repeat the weakest 2-minute section and log the next practice target	Before/after attempt and Notion record	OBS + Notion

Do not spend the entire 100 minutes reviewing the video. Your highest-value cycle is **prepare -> predict -> verify -> teach -> inspect -> correct -> repeat**.

9. Teaching-performance rubric

Score each dimension from 1 to 4 after the session. These are personal coaching indicators, not a validated psychometric instrument.

Dimension	How the 1-4 progression should be judged
Technical correctness	**1:** key rule wrong or missing. **2:** common cases correct, edge cases weak. **3:** correct within intended scope. **4:** correct, precise, and carefully qualified at boundaries.
Conceptual explanation	**1:** terminology listed without relationships. **2:** example given but reasoning skipped. **3:** rule, example, and output connected. **4:** anticipates misconceptions and explains why alternatives fail.
Code trace	**1:** reads code without predicting. **2:** traces main path with prompting. **3:** predicts and explains output. **4:** handles variants and diagnoses wrong predictions.
Teaching sequence	**1:** prerequisites or transitions unclear. **2:** mostly ordered, but jumps remain. **3:** clear progression from simple to complex. **4:** makes learner thinking visible and checks each key step.
Delivery	**1:** difficult to follow or note-dependent. **2:** uneven pace or board orientation. **3:** clear and appropriately paced. **4:** uses pauses, emphasis, board, and questioning deliberately.
Evidence of learning	**1:** no understanding check. **2:** recall-only question. **3:** prediction or application question. **4:** a new problem shows transfer beyond the example.

Readiness gate: never call the concept ready to teach if you have an unresolved technical error. Once the code and rules are sound, prioritize teaching clarity and learner transfer rather than polishing the wording endlessly.

10. Active-recall and viva questions

Attempt each question aloud before consulting an answer. For code questions, write down your prediction before running the program.

Basic

1. Why is `Object` called the root of the ordinary Java class hierarchy?
2. What is the difference between a variable's declared type and an object's runtime class?
3. What does `toString()` do, and why is its default output not a memory address?
4. Are constructors inherited by subclasses?
5. What does a `final` class prevent?
6. What does a `final` method prevent?
7. What does `final` on a reference variable prevent?

Intermediate

8. Why might `equals(Ticket)` fail to override `Object.equals(Object)` even if its body compares ticket numbers?
9. If a class overrides `equals`, why is it generally necessary to override `hashCode`?
10. Why is “private members are inaccessible to subclasses” not the same statement as “private state is absent from subclass instances”?
11. Why does package-private access not cross into a subclass merely because it is a subclass?

12. In another package, why might this.protectedField be legal inside subclass code while baseReference.protectedField is illegal?
13. Can a final class be instantiated? Explain.
14. Can a final reference refer to an object whose mutable field changes? Explain.

Advanced

15. A class defines boolean equals(Order other). Predict what happens when an Object-typed reference to the same instance is compared using equals against another Object reference. Explain overload resolution versus overriding.
16. Two objects are not equal but return the same hash code. Is this a contract violation? Why?
17. A method is final. Does the Java language guarantee that the runtime inlines it? Why not?
18. Why can a superclass method read its own private field while subclass code cannot refer to that field as an inherited member?
19. Explain the cross-package protected receiver rule without using the phrase “protected means visible to children.”
20. Which parts of the statement “final makes code safer and faster” are language guarantees, design judgments, or unsupported generalizations?

Suggested answer key

1. Ordinary classes inherit directly or indirectly from Object unless their superclass is explicitly specified; Object has no superclass.
2. The declared type controls which members are available to source-level compile-time checks; runtime class identifies the actual object class.
3. It returns a textual representation. The base representation is class-name plus hash-code-related text, not a guaranteed address.
4. No. Constructors initialize instances but are not inherited members.
5. Subclassing.
6. Overriding or hiding that method by a subclass.
7. Reassigning the variable after assignment; it does not by itself prevent mutation of the referenced object.
8. equals(Ticket) has a different parameter type from equals(Object), so it is an overload, not an override.
9. Equal objects must have equal hash codes for the hash-based collection contract to work correctly.
10. Private fields are not inherited as subclass members, but superclass object state and superclass methods still exist as part of normal object construction and behavior.
11. Package access is limited by package membership; inheritance does not erase the package boundary.
12. In cross-package subclass code, protected instance-member access is constrained by the receiver's qualifying type: it must be the subclass type or a subtype under the JLS rule.
13. Yes, provided it has an accessible constructor. final blocks subclassing.
14. Yes. The reference cannot be reassigned, but mutable state of the referenced object can change if an allowed operation changes it.
15. The expression is resolved using the compile-time types and available signatures. An Object-typed call selects equals(Object); an equals(Order) overload is not the override.
16. No. Unequal objects are allowed to share a hash code; the required implication is equality -> same hash code.
17. No. The implementation may optimize/in-line as long as semantics are preserved, but the keyword does not mandate a particular optimization.
18. The declaration's private access belongs to the declaring class context; inheritance and private access are distinct language concepts.
19. Access outside the package must occur within subclass code, and for an instance field/method, the qualifying expression must have the subclass type or a subtype.

20. final imposes particular language restrictions. It can express an API/design boundary; “always faster” is not a Java language guarantee.

11. Deliberate lab experiments

For each experiment: predict first, state the rule, compile or run, compare output/diagnostic, and explain the difference.

1. **Runtime class:** store a Printer in an Object reference, call getClass(), and explain why printPage() is still unavailable through that Object-typed expression.
2. **`toString`:** print an object before and after overriding toString(); list exactly what the override intentionally exposes.
3. **`equals`:** compare two distinct values with default equality, then implement an exact equals(Object) override and a consistent hashCode().
4. **Package access:** move the subclass into a different package; predict which package-private member references stop compiling.
5. **Protected receiver:** compare this.value, Child other.value, and Base base.value inside a cross-package subclass.
6. **Final class:** instantiate a final class, then separately attempt to extend it in an invalid experiment.
7. **Final method:** call a final method normally, then separately attempt an override in an invalid experiment.
8. **Final reference:** mutate an object's field through a final reference, then separately attempt reference reassignment.

Keep deliberately invalid code outside the set of files used in a compile-all command. A clean compile of the valid examples should remain meaningful.

12. Source hierarchy and audit notes

Use the sources in this order when they disagree:

1. **JLS** for language legality and semantics.
2. **Java API documentation** for the contract of methods such as Object.equals, hashCode, toString, and getClass.
3. **Dev.java** for the readable narrative and teaching sequence.
4. **Existing faculty guides and books** for local continuity, examples, and teaching ideas, subject to verification.

Key references

[1] Oracle, *Java Language Specification, Java SE 27*, Chapter 8: Classes, especially §§8.1.4, 8.2, 8.3, 8.4.3.3, and 8.4.8. https://docs.oracle.com/javase/specs/jls/se27/html/jls-8.html

[2] Oracle, *Java Language Specification, Java SE 27*, Chapter 6: Names, especially §§6.6.1, 6.6.2, and 6.6.2.1 for accessibility and protected access. https://docs.oracle.com/javase/specs/jls/se27/html/jls-6.html

[3] Oracle, *Java SE 27 API: `java.lang.Object`*, including the contracts for equals, hashCode, toString, getClass, clone, and lifecycle methods. https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/lang/Object.html

[4] Dev.java, *Inheritance*: readable explanation of superclass/subclass relationships and inherited members. https://dev.java/learn/inheritance/what-is-inheritance/

[5] Existing local resource: *OOPJ Unit III: Inheritance Fundamentals, `Object`, Inherited-Member Access, and `final`*, Integrated Faculty Study Guide, Chapter 15. Use it as the broader inheritance context and this guide as a focused practice companion.

Faculty accuracy checkpoints

Before using these materials in class, be able to say each of the following without hesitation:

- Object.toString() is not a guaranteed memory-address printer.
- equals(T) is not an override of equals(Object).
- Equal objects must have equal hash codes; unequal objects may share a hash code.
- Inheritance and accessibility are connected but distinct rules.
- Constructors are not inherited.
- Cross-package protected instance-member access depends on the qualifying type.
- final means different things depending on whether it modifies a class, method, or variable.
- A final reference does not imply deep immutability.
- JIT inlining is not a Java language guarantee implied by final.

Library resource map and honest scope audit

This guide integrates the topic-specific visuals directly into the reading flow and carries forward selected prerequisite visuals from Unit II. The audit below is deliberately scoped to resources relevant to Object, access control, inheritance and finality. It is not a claim that every unrelated item in the entire Library has been read or every slide image in every deck has been inspected.

Resource	Contribution	Use status / limitation
OOPJ_Unit_III_Inheritance_Fundamentals_Faculty_Guide (.md/.docx)	Main inheritance baseline: hierarchy, Object, inherited members, protected nuance, final, design choices and misconceptions.	Reviewed and used as the main topic baseline; this guide deepens three areas rather than replacing inheritance as a whole.
OOPJ_Unit_III_Research_Curation_Blueprint.md	Unit III scope, sequence, source hierarchy and research quality gates.	Reviewed for scope and the planned progression.
OOPJ_Concept_2_Class_Declaration_Access_Control_Encapsulation_Faculty_Reference (.docx/.pdf/.md)	Prerequisite access levels, class/member distinction and encapsulation.	Consulted for continuity and terminology; not duplicated in full.
OOPJ_Unit_II_Remaining_Topics_Faculty_Study_Guide (.docx/.pdf/.md)	Prerequisites around references, final, method signatures and overriding.	Consulted selectively for cross-unit connections.
OOPJ_Concept_3 study guide, source audit and Figure 3.1–3.3 PNG/SVG resources	Instance/static members; declaration versus creation; aliasing, mutation and reassignment.	Three PNG visuals are embedded at the start of this guide. Their editable counterparts and existing Miro links are indexed in the resource map.
Think Java, 2nd edition, Chapter 14	Readable inheritance teaching sequence and class-extension examples.	Used selectively; the equals(Time) example is an overload, not an override of equals(Object).
OOPJ(5).pdf lab manual	Local lab scope around inheritance and related Java exercises.	Used as a scope/skill alignment reference, not as authority for Java semantics. Duplicate OOPJ PDFs were not each audited independently.
unit-1-JP.pdf and Java_Architecture_and_Memory_Dynamics.pptx	Contextual OOP / JVM background.	unit-1-JP used only as contextual background. Extracted text from the 21-slide architecture deck did not establish direct support for these rules; its visuals were not exhaustively audited.
JLS Java SE 27, Chapters 6 and 8; Java SE Object API	Authoritative source for access, inheritance, finality and Object method contracts.	The guide uses these as the technical authority; citations and URLs are included in the reference section.
Dev.java, Helsinki Java MOOC, NPTEL / Debasis Samanta	Alternative explanations and pedagogical sequencing.	Used selectively for teaching ideas; tutorial simplifications and older examples do not override the JLS.

Existing prerequisite Miro resources: the board is named GPCET OOPJ Knowledge Graph. The three links below open the relevant widgets. These are Unit II foundation visuals, not a pre-existing inheritance-specific board.

- Figure 3.1 - Class members: https://miro.com/app/board/uXjVHkEzizI=/?moveToWidget=3458764686602462953
- Figure 3.2 - Declaration versus creation: https://miro.com/app/board/uXjVHkEzizI=/?moveToWidget=3458764686602462954
- Figure 3.3 - Aliasing, mutation and reassignment: https://miro.com/app/board/uXjVHkEzizI=/?moveToWidget=3458764686602462955

The five topic-specific figures embedded in the main text have corresponding PNGs for viewing/printing and editable SVGs in the accompanying visual package: (1) Object hierarchy and methods; (2) inherited-member access map; (3) protected cross-package receiver rule; (4) final class/method/variable distinctions; (5) Object methods and equals signature trap.

Runnable Java example map

Extract OOPJ_Unit_III_Object_Access_Final_Examples.zip. Keep valid examples separate from the intentionally invalid examples so a compile-all command does not fail by design.

File	Focus	How to use
RuntimeClassDemo.java	Object reference to a subclass; getClass() versus declared reference type.	Valid; run and compare output.
ObjectMethodsDemo.java	toString(), equals(Object), hashCode() consistency and the overload/override distinction.	Valid; use output to explain API behaviour.
InheritedMembersDemo.java	Inherited versus declared members; accessible API versus private field access.	Valid; predict which calls compile.
ProtectedAccessDemo.java + alpha/Base.java	Protected access in a real two-package layout; legal and illegal receivers.	Valid core example; use separate negative example to test boundary.
FinalRulesDemo.java	Final class, final method and final reference effects.	Valid; contrast permitted mutation with prohibited reassignment.
invalid-examples/*.java	FinalClassBad, FinalMethodBad, FinalReferenceBad, ProtectedReceiverBad.	Intentionally invalid; compile one at a time and explain expected diagnostic.

Run on Windows with run_examples.bat or run_examples.sh on a compatible shell. Check the README and VALIDATION_RECORD in the extracted package. The code package keeps invalid snippets outside the successful-run source set.

Five topic visuals: quick retrieval prompts

- Figure 1 - Object hierarchy: Explain why an Object reference can refer to a Printer, yet cannot directly call a Printer-only method.
- Figure 2 - Inherited-member access map: In what order should you check membership, accessibility, and the reference type?
- Figure 3 - Protected receiver rule: In another package, why can Child access this.score or child.score but not base.score when base is declared Base?
- Figure 4 - final distinctions: What is prohibited for a final class, final method, and final reference? What remains permitted?
- Figure 5 - Object methods/equality: Why is equals(Ticket) an overload while equals(Object) is the override signature?

13. Completion checklist

Mark this chapter ready for teaching only when you can do the following without looking at the guide:

- [ ] Draw and explain the Object -> superclass -> subclass relationship.
- [ ] Explain and demonstrate toString, equals(Object), hashCode, and getClass.
- [ ] Identify an equals(T) overload trap.
- [ ] Distinguish inherited membership from accessibility at a particular code location.
- [ ] Explain why constructors and private superclass fields are not inherited.
- [ ] Predict cross-package protected-access legality and justify the receiver rule.
- [ ] Explain final class, final method and final reference with separate examples.
- [ ] Compile and run every valid example and explain the invalid examples' diagnostics.
- [ ] Teach the core distinctions in a short recording without reading full paragraphs.
- [ ] Use a new example to check learner transfer, not just definition recall.

Next step in Unit III: once these focused inheritance foundations are secure, continue with Chapter 16 in the existing blueprint: constructor chaining and super, then proceed to overriding/dynamic dispatch and abstract classes before beginning the interface block. Arrays remain part of the unit-level syllabus, but they are not the focus of this requested guide.

My concept mastery and teaching-improvement log

Complete this after a study/rehearsal attempt. Record evidence, not just a confidence feeling. Use one copy per concept or practice date.

Reflection prompt	My evidence / notes
Concept and date	_____
Rule I can now explain from memory	_____
Code prediction and verified result	_____
Misconception or edge case I corrected	_____
Delivery issue noticed in recording	_____
Next concrete improvement to test	_____